

Programação Imperativa — Teste, 27 de Maio de 2026

1. Strings (3 valores)

Defina uma função `int parenthesis_ok (char exp[])` que, dada uma string onde está armazenada uma expressão aritmética com parêntesis, testa se os parêntesis estão corretos. Por exemplo, se a expressão for "31 + ((2 - 13) - x) + 114" a função deve retornar verdadeiro (1), enquanto que para a expressão "31 + (2 - (13 + 4))) + ((2 - x) + 114" deve retornar falso (0).

2. Arrays (3 valores)

Assuma que existe uma função `int random(int N)` que devolve um número aleatório entre 0 e N-1. Defina uma função `void sorteia(int a[], int N)` que, dado $N > 0$, preenche o array a, de tamanho N, com todos os números de 0 a N-1, colocados em posições aleatórias. No final, cada um desses números deve aparecer exatamente uma vez no array. Por exemplo, se $N == 5$, o array poderá ficar com os valores {3, 0, 4, 1, 2} ou {1, 4, 2, 0, 3}, mas não poderá ficar com {1, 1, 2, 3, 4}. Tente minimizar o número de invocações à função random.

3. Ordenação de arrays (1+2=3 valores)

1. Defina a função `int minInd (int v[], int N)` que, dado um array v com N elementos ($N > 0$) calcula um índice do array onde está armazenado o menor elemento do array v.
2. Defina a função `void minSort (int v[], int N)` que ordena um array de N inteiros por sucessivas chamadas à função minInd.

4. Listas Ligadas (3 valores)

Defina uma função `int quantosMaiores (LInt l)` que, dada uma lista **não vazia**, calcula quantas vezes ocorre na lista o maior elemento.

```
typedef struct no {  
    int valor;  
    struct no *prox;  
} *LInt;
```

5. Ordenação de Listas (3+2=5 valores)

1. Defina a função `LInt retiraMaior (LInt *l)` que retira da lista `*l` o nodo onde se encontra o maior elemento da lista. A função retorna o endereço do nodo removido.
2. Defina a função `LInt maxSort (LInt l)` que ordena uma lista por sucessivas chamadas à função `retiraMaior`.

6. Árvores binárias (3 valores)

Relembre a função `void mirror (ABin a)` que inverte uma árvore binária. Uma árvore diz-se simétrica se for igual à sua inversa. Defina a função `int simetrica (ABin a)` que testa se uma árvore é simétrica (por exemplo, a árvore apresentada abaixo é simétrica).

```
typedef struct nodo {  
    int valor;  
    struct nodo *esq, *dir;  
} *ABin;
```

